

Backend (Node.js)

Most Frequently Asked Interview Questions

READ MORE



Core Node.js

1. What is Node.js? Why is it considered single-threaded?
2. Explain the Node.js Event Loop and its phases.
3. Difference between `process.nextTick()`, `setImmediate()`, and `setTimeout()`.
4. What are Streams in Node.js? Explain Readable, Writable, Duplex, and Transform streams.
5. What is EventEmitter? Give a real-world use case.
6. How does Node.js handle concurrency despite being single-threaded?
7. Difference between synchronous and asynchronous APIs.
8. Explain callbacks, Promises, and `async/await`.
9. What are Worker Threads? When should you use them?
10. What is the Cluster module? How does it utilize multiple CPU cores?
11. How does Node.js manage memory? What causes memory leaks?
12. CommonJS vs ES Modules.
13. What are Buffers in Node.js?

Core Node.js

13. What are Buffers in Node.js?

14. How do you handle errors in asynchronous Node.js code?

15. How would you debug a slow or memory-intensive Node.js application?

Express.js

1. Explain the request-response lifecycle in Express.
2. What are middleware functions? How do they work?
3. Explain application-level, router-level, built-in, and error-handling middleware.
4. How do you implement JWT-based authentication?
5. JWT vs Session-based authentication. When would you use each?
6. How do Access Tokens and Refresh Tokens work?
7. How do you implement authorization and Role-Based Access Control (RBAC)?
8. How do you validate and sanitize incoming request data?
9. How do you handle file uploads using Multer?
10. How do you implement centralized error handling?
11. How do you secure an Express API?
12. How do you implement API rate limiting?
13. How do you implement logging in a production application?
14. How would you structure a scalable Node.js/Express application?

Database (SQL / NoSQL)

1. SQL vs NoSQL. When should you choose each?
2. Explain database normalization and denormalization.
3. What are Primary Keys, Foreign Keys, and Unique Keys?
4. What are database indexes? How do they improve query performance?
5. INNER JOIN vs LEFT JOIN vs RIGHT JOIN vs FULL JOIN.
6. What are transactions? Explain ACID properties.
7. What is database connection pooling? Why is it important?
8. How do you identify and optimize slow SQL queries?
9. Explain one-to-one, one-to-many, and many-to-many relationships.
10. How do you prevent SQL Injection?
11. Explain the MongoDB Aggregation Pipeline.
12. Embedding vs Referencing in MongoDB.
13. Offset pagination vs Cursor-based pagination.
14. Explain Optimistic vs Pessimistic Locking.
15. What is the N+1 Query Problem? How do you prevent it?

REST APIs & System Design

1. What are REST principles?
2. Difference between POST, PUT, and PATCH.
3. What is idempotency? Which HTTP methods are idempotent?
4. What HTTP status codes should every backend developer know?
5. How do you design scalable REST APIs?
6. How do you implement API versioning?
7. How do you document APIs using Swagger/OpenAPI?
8. What is CORS? How does it work?
9. What is an API Gateway? Why is it used?
10. How does Redis caching work?
11. Explain common caching strategies: Cache-Aside, Write-Through, Write-Behind, and TTL.
12. What are WebSockets? When would you use them instead of HTTP?
13. What are Message Queues? Explain RabbitMQ/Kafka use cases.

REST APIs & System Design

- 14. How do retries, exponential backoff, and dead-letter queues work?
- 15. How would you implement rate limiting for a public API?
- 16. What is load balancing? How does it improve scalability?
- 17. How would you design a URL Shortener, Notification Service, or Real-Time Chat system?

Backend Performance & Security

1. How do you optimize the performance of a Node.js application?
2. How do you detect and prevent memory leaks?
3. What is Event Loop blocking? How do you avoid it?
4. How would you handle high traffic and concurrent requests?
5. Horizontal Scaling vs Vertical Scaling.
6. What is load balancing? Explain common load-balancing strategies.
7. How should passwords be securely stored?
8. Hashing vs Encryption. What is the difference?
9. What is bcrypt? Why is password salting important?
10. How do you prevent XSS, CSRF, and SQL/NoSQL Injection?
11. How do Access Tokens and Refresh Tokens work securely?
12. What is Helmet.js? What security headers does it help configure?
13. What are Docker containers, images, volumes, and networks?
14. How would you deploy and scale a Node.js backend in production?

Some additional topics :

- Redis and cache invalidation
- Database replication and read replicas
- Database sharding and partitioning
- Reverse proxy and Nginx
- Microservices vs Monolith
- API idempotency keys
- Webhooks
- Background jobs and job queues
- Distributed locks
- Graceful shutdown
- Health checks
- Docker and CI/CD
- AWS basics (EC2, S3, RDS, Lambda)
- Observability: Logs, Metrics, and Tracing